PERSONALIZED LEARNING & CAREER AGENT

Course Notes for a Retrieval-Augmented Generation (RAG) Study Companion

Purpose: These notes are a small, self-contained sample knowledge base for testing PDF loading, text splitting, embeddings, Chroma retrieval, and question answering in a LangChain project.

Contents

- 1. Introduction to AI Study Companions
- 2. Retrieval-Augmented Generation (RAG)
- 3. Document Loading and Text Splitting
- 4. Embeddings and Vector Databases
- 5. Retrievers and Retrieval Quality
- 6. Tools and Agent Routing
- 7. Learning Resources and Job Matching
- 8. Evaluation and Responsible Use
- 9. Glossary and Practice Questions

1. Introduction to AI Study Companions

An AI study companion is an application that helps a learner understand course material, revise concepts, and discover learning or career opportunities. A useful companion should distinguish between answers that must come from course notes and requests that need current external information.

For example, the question “What is a vector database?” should be answered using the course knowledge base when relevant notes are available. The question “Find current machine-learning internships” requires an external job source or API because listings and deadlines change.

A well-designed system can support three request types:

- **Course-content request:** retrieve relevant course passages and explain them clearly.
- **Opportunity-search request:** search an external source for learning resources, jobs, or internships.
- **Combined request:** explain a concept using course notes and then find opportunities related to that topic.

2. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a method that combines information retrieval with text generation. Before answering a question, the application searches a collection of documents for relevant passages and supplies those passages to a language model as context.

A typical RAG workflow is:

- Load source documents, such as lecture notes or a textbook chapter.
- Split the documents into manageable text chunks.
- Convert each chunk into an embedding, which is a numerical representation of its meaning.

- Store embeddings and associated text in a vector store.
- Retrieve chunks that are semantically related to a user's question.
- Ask the language model to answer using the retrieved context.

Example: If a learner asks “Why do we split documents before indexing?”, the retriever can find the section describing chunking. The language model then explains that smaller passages make retrieval more focused and help fit useful context into a prompt.

RAG can make answers more grounded in a chosen source, but it does not guarantee correctness. Retrieval can miss relevant passages, source documents can be incomplete, and a language model can still misunderstand the retrieved context.

3. Document Loading and Text Splitting

Document loading extracts content from source files and represents it as documents that a framework can process. In LangChain, PyPDFLoader can load a PDF. A text loader can be used for plain-text files.

With PyPDFLoader, each PDF page is commonly returned as a Document object. Its page_content field contains extracted text, while metadata can include details such as the source file and page number.

Text splitting divides long text into smaller chunks. RecursiveCharacterTextSplitter tries separators in a preferred order so that it can split text at natural boundaries when possible.

Important splitter settings:

Setting	Meaning
chunk_size	Approximate maximum size of each chunk, measured in characters when using the default length function.
chunk_overlap	Amount of shared text between neighboring chunks to preserve context across boundaries.
separators	Preferred boundaries such as paragraphs, lines, spaces, or individual characters.

Chunk size and overlap should be tested on the actual course material. Very large chunks may contain unrelated topics; very small chunks may lose context. Overlap can help when a definition or explanation crosses a chunk boundary.

4. Embeddings and Vector Databases

An embedding model maps text to a vector of numbers. Texts with related meanings often have vectors that are close under a similarity measure. This makes semantic search possible even when the user's wording differs from the exact wording in the notes.

A vector store keeps vectors together with the original text and metadata. Chroma is one option that supports local vector storage and persistence. Persisting the database means the application can reuse the indexed collection after the program restarts, instead of embedding every document again.

For a local project, choose an embedding model that is compatible with the installed libraries and available on the machine. The chat model and the embedding model perform different jobs: the chat model generates answers, while the embedding model represents text for retrieval.

5. Retrievers and Retrieval Quality

A retriever accepts a query and returns documents that appear relevant. A vector-store retriever commonly compares the query embedding with stored chunk embeddings. The returned chunks are then passed to the language model as context.

Retrieval should be tested with more than one style of question:

- **Narrow factual question:** “What does chunk_overlap mean?”
- **Conceptual question:** “Explain how RAG works from PDF to final answer.”
- **Paraphrased question:** “Why do adjacent text pieces share some content?”
- **Out-of-scope question:** ask something that is not discussed in the PDF and check whether the system admits the source does not contain the answer.

Review the retrieved text itself, not only the final answer. If the correct passage is missing, improve the source material, chunking, embedding choice, or retrieval settings before changing the prompt.

A grounded answer should clearly separate information found in the notes from any general explanation added by the model.

6. Tools and Agent Routing

An agent can choose and call tools to complete a request. In a learning-and-career assistant, two useful tools are a course-material retrieval tool and an opportunity-search tool.

The course retrieval tool should return relevant passages from the indexed notes. Its description should explicitly say that it is for answering questions about course content.

The opportunity-search tool should query an external job or learning-resource source using topic keywords. Its description should explicitly say that it searches external opportunities rather than explaining course material.

A routing prompt can instruct the agent to classify each request as a course question, an opportunity search, or both. For opportunity results, the agent should explain the match using evidence such as required skills, role description, level, or topic—not just repeat a job title.

Tool descriptions should be specific and non-overlapping. The agent should not invent current openings if the external search returns no results, and should not present a course-note passage as a live job listing.

7. Learning Resources and Job Matching

Opportunity matching maps what a student is studying to the skills and topics mentioned in a learning resource, job, or internship. A basic implementation can extract keywords from the user's topic and compare them with each result's title, description, and skills.

Example: A student studying Python, machine learning, and retrieval-augmented generation may match an entry-level AI internship that mentions Python, NLP, vector search, or LLM applications. The match should be described cautiously and should not imply that the student meets every requirement.

A simple matching process:

- Collect the student's studied topics or skills.
- Search an API or dataset using one or more relevant keywords.
- Normalize text and compare skills and topics against each result.
- Rank results using transparent matching rules.
- Show the title, organization, location or remote status when available, source link, and a short explanation of the match.

- Clearly label missing fields and avoid inventing salary, deadlines, or eligibility details.

For live listings, use a current API or reliable source and respect its access limits. A static sample dataset is useful for development but should not be described as real-time information.

8. Evaluation and Responsible Use

Evaluate the project in separate stages so that failures are easier to diagnose.

- **LLM test:** verify that the local model returns a short answer.
- **Loading test:** confirm that PDF pages and readable text are extracted.
- **Splitting test:** inspect several chunks and verify that explanations are not cut awkwardly.
- **Retrieval test:** check that the correct section appears for factual and conceptual questions.
- **Agent test:** verify that course questions call the retrieval tool and opportunity questions call the external search tool.
- **Interface test:** verify errors, empty results, and long responses are handled clearly.

The assistant should protect user information, avoid exposing private documents to external services without permission, and make it clear when answers are based on course notes versus external sources. Job and internship details can change, so users should verify the original listing before applying.

9. Glossary

LLM: Large Language Model; a model that processes and generates text.

RAG: Retrieval-Augmented Generation; retrieve source context and use it to help generate an answer.

Loader: A component that reads a source file and converts it into processable documents.

Chunk: A smaller piece of a document used for indexing and retrieval.

Embedding: A numerical vector representation of text.

Vector store: A system that stores vectors and associated content for similarity search.

Retriever: A component that returns documents relevant to a query.

Tool: A callable function an agent can use to access a capability or data source.

Agent: A system that decides which tools to use and how to combine their results.

Persistence: Saving an index or database so it can be reused later.

Practice Questions

- 1. Define Retrieval-Augmented Generation (RAG).
- 2. Why is a long PDF split into chunks before embedding?
- 3. Explain chunk_size and chunk_overlap.
- 4. What is an embedding, and how does it support semantic search?
- 5. What is the role of Chroma in a RAG application?
- 6. How would you test whether a retriever answers conceptual questions well?
- 7. Explain the difference between a course retrieval tool and an opportunity-search tool.
- 8. How can an agent explain why an internship matches a student's topics?

- 9. Why should a system not invent job listings when search results are empty?
- 10. List four stages at which a RAG study companion should be evaluated.

End of sample course notes

This PDF is intentionally written as sample course material for a student project. You can replace or supplement it later with your own syllabus, lecture notes, or a public textbook chapter.